

Pipeline RAG d'analyse argumentative

Guide opérationnel — mise en œuvre et test

Version avril 2026 — mis à jour le 19 avril 2026

Ce document décrit les deux pipelines (OpenAI et local Ollama) et les étapes de mise en œuvre pour une nouvelle installation.

1. Vue d'ensemble

Le pipeline est organisé en deux voies indépendantes qui partagent la même logique mais diffèrent sur le moteur LLM et la gestion des données.

Dimension	Voie OpenAI	Voie locale (Ollama)
LLM de génération	gpt-4.1-mini ou gpt-4.1	qwen2.5:14b, deepseek-r1:32b, gemma3:12b...
Embeddings	text-embedding-3-large (API)	paraphrase-multilingual-mpnet-base-v2 (local)
Clé API requise	Oui (.env)	Non
Données transmises	Paragraphes + passages corpus	Aucune — tout reste en local
Coût	~0.10–0.50 \$ / run batch	Zéro
RAM requise	Minimale	16 Go (14B) / 20 Go (32B)
Qualité batch 09/10	Très bonne	Bonne à excellente (32B recommandé)
Qualité segments A/B/C/D	Excellente	Bonne à très bonne selon le modèle

⚠ Les deux voies partagent les mêmes scripts 11, A, B, C, D (suffixe `_local` pour Ollama). Les scripts 01, 02, 03 sont identiques dans les deux voies.

2. Prérequis communs aux deux voies

2.1 Environnement Python — Apple Silicon

⚠ Anaconda installe par défaut une version Intel (x86_64) sur Mac. Utiliser Miniforge, nativement arm64.

```
curl -L https://github.com/conda-  
forge/miniforge/releases/latest/download/Miniforge3-MacOSX-arm64.sh -o  
miniforge.sh  
bash miniforge.sh  
# Ouvrir un nouveau terminal, puis :  
python -c "import platform; print(platform.machine())" # → arm64  
  
conda create -n rag_historien python=3.11  
conda activate rag_historien  
pip install sentence-transformers faiss-cpu numpy requests python-dotenv  
pip install pdfminer.six tiktoken weasyprint openai pymupdf
```

- ⚠ Sur macOS, si WeasyPrint échoue : `brew install pango cairo`
- ⚠ Ne jamais travailler dans l'environnement base. Toujours activer `rag_historien`.

2.2 Structure du dossier projet

```

MonProjet/
├── .env                ← OPENAI_API_KEY=sk-... (voie OpenAI uniquement)
├── rag_config_local.py ← configuration voie locale
├── config_00.py        ← configuration voie OpenAI
├── [tous les scripts .py]
├── pdfs/              ← vos PDFs corpus
├── extracted_text/    ← produit par 01 et 02
├── vector_store/      ← produit par 03 (voie OpenAI)
├── vector_store_local/ ← produit par 03_local (voie Ollama)
└── outputs/           ← toutes les sorties (anciennement resultats/)

```

2.3 Voie locale — Ollama

```

# Installer Ollama : https://ollama.com
ollama serve                # lancer en arrière-plan
ollama pull qwen2.5:14b     # modèle par défaut

# Sur Mac M2/M3 avec 32 Go — modèles 32B disponibles :
ollama pull deepseek-r1:32b # recommandé pour 09/10/A/B/C (~20 Go)
ollama pull qwen3:32b      # excellent en français académique

# Alternatives 14B (16 Go) :
ollama pull deepseek-r1:14b
ollama pull gemma3:12b

```

- ⚠ Modifier `LLM_MODEL` dans `rag_config_local.py` pour changer de modèle sur tout le pipeline.

3. Pipeline local (Ollama)

3.1 Voie A locale — avec Zotero

Étape	Commande
Zotero → CSV + HTML	<code>python 00a_html_to_pdf.py && python 00_zotero_import.py</code>
Extraction + chunking	<code>python 01_extract_text.py && python 02_chunk_corpus.py</code>
Embeddings locaux	<code>python 03_build_embeddings_local.py</code>
Confrontation paragraphe	<code>python 05_confronter_local.py --paragraphe mon_para.txt</code>
Cartographie enrichie	<code>python 06_map_enrich_local.py --manuscrit manuscript.txt</code>
Cartographie critique	<code>python 07_map_critique_local.py outputs/map_{ts}.json</code>
Argumentation batch	<code>python 09_map_argumentation_local.py</code>
Perelman batch	<code>python 10_map_perelman_local.py</code>
Zones prioritaires	<code>python 11_zones_prioritaires.py</code>
Analyse segment A	<code>python A_toulmin_segment_local.py --fichier segment.txt</code>
Analyse segment B	<code>python B_perelman_segment_local.py --fichier segment.txt</code>
Analyse segment C	<code>python C_rst_segment_local.py --fichier segment.txt</code>

Synthèse croisée D	python D_synthese_croisee_local.py
--------------------	------------------------------------

3.2 Voie B locale — sans Zotero

Déposer les PDFs dans pdfs/ et sauter les étapes 00a et 00_zotero_import. Les références affichées seront les noms de fichiers.

4. Choix du modèle LLM local

Modifier LLM_MODEL dans rag_config_local.py — une seule ligne suffit.

Modèle	RAM	Usage recommandé
deepseek-r1:32b	~20 Go	Recommandé sur 32 Go — scripts 09, 10, A, B, C
qwen3:32b	~20 Go	Recommandé sur 32 Go — excellent en français académique
deepseek-r1:14b	16 Go	Scripts 09, 10, A, B, C — raisonnement structuré
gemma3:12b	16 Go	Scripts 09, 10, A, B, C — très bon en français
qwen2.5:14b	16 Go	Défaut — bon équilibre général sur tous les scripts
mistral:7b-instruct	8 Go	Scripts 04, 05, 06, 07 — si RAM < 16 Go
qwen2.5:7b	8 Go	Si RAM < 16 Go — qualité correcte pour 06/07

5. Paramètres importants (rag_config_local.py)

Paramètre	Valeur recommandée	Rôle
LLM_MODEL	deepseek-r1:32b	Modèle Ollama pour tout le pipeline
LLM_TEMPERATURE	0.1	Déterminisme des réponses
LLM_MAX_TOKENS	2048 (4500 pour A/B, 5000 pour C)	Tokens max en sortie
TOP_K	6	Passages corpus par paragraphe (06, 07)
FAISS_INDEX_TYPE	flat	Type d'index (flat recommandé < 50 000 chunks)

6. Paramètres spécifiques par script

Script 06 — map_enrich_local.py

Paramètre	Valeur	Note
TOP_K_MAP	6	Passages par section (défaut 25 trop élevé)

AUTO_SEGMENT	False	Segmentation par paragraphes (True = LLM, moins précis sur exports Word)
PARAGRAPHE_MIN_CHARS	150	Filtre les en-têtes de page et titres isolés
MAX_SECTIONS	20 (None = toutes)	Limiter pour tester

Script 11 — zones_prioritaires.py

⚠ Vérifier `JSON_DIR = "outputs"` en tête du script (certaines versions pointent vers "resultats").

7. Corrections apportées aux scripts (avril 2026)

Scripts A, B, C, D — import incomplet

Les scripts locaux avaient un import incomplet depuis `rag_config_local`. Correction appliquée à tous :

```
from rag_config_local import (
    LLM_MODEL, OLLAMA_URL,
    LLM_MAX_TOKENS as MAX_TOKENS,
    LLM_TEMPERATURE as TEMPERATURE,
)
OUTPUT_DIR = "outputs"
```

Scripts 09 et 10 — extraction des scores

Portage de la fonction `extraire_scores()` à 4 niveaux depuis la version OpenAI. Sans cette correction, tous les scores tombent à 0.50 (valeur neutre par défaut).

Script 06 — segmentation du manuscrit

- `AUTO_SEGMENT = True` → passer à `False` pour manuscrits non structurés
- `text.split("\n\n")` → `re.split(r"\n\s*\n", text)` (regex flexible)
- Filtre < 50 chars → < 150 chars (élimine en-têtes de page Word/PDF)

8. Procédure de test (voie locale)

8.1 Vérifications initiales

Test	Commande	Résultat attendu
Architecture	<code>python -c "import platform; print(platform.machine())"</code>	arm64
Ollama actif	<code>curl http://localhost:11434</code>	<code>{"ollama": "is running"}</code>
Modèle disponible	<code>ollama list</code>	deepseek-r1:32b listé
FAISS	<code>python -c "import faiss; print(faiss.__version__)"</code>	1.9.0
sentence-transformers	<code>python -c "import sentence_transformers; print('ok')"</code>	ok

8.2 Test du pipeline complet

Étape	Commande	Sortie attendue
Embeddings	python 03_build_embeddings_local.py	vector_store_local/ créé
Requête	python 04_rag_query_local.py	Réponse streamée
Cartographie	python 06_map_enrich_local.py --manuscrit manuscript.txt	map_{ts}.json
Critique	python 07_map_critique_local.py outputs/map_{ts}.json	critique.json
Argumentation	python 09_map_argumentation_local.py	argumentation_{ts}.json
Perelman	python 10_map_perelman_local.py	perelman_{ts}.json
Zones	python 11_zones_prioritaires.py	zones_{ts}.md
Segment A	python A_toulmin_segment_local.py --fichier segment.txt	toulmin_{ts}.md

9. Problèmes fréquents et solutions

Symptôme	Cause probable	Solution
Scores tous à 0.50	LLM n'a pas respecté le format X/10	Utiliser les scripts corrigés (avril 2026) avec extraire_scores() à 4 niveaux
NameError: OUTPUT_DIR / MAX_TOKENS / TEMPERATURE	Import incomplet dans A/B/C/D	Appliquer la correction d'import (section 7)
Seulement 5-6 sections dans le 06	AUTO_SEGMENT = True ou regex rigide	AUTO_SEGMENT = False + regex flexible (section 6)
FileNotFoundError: argumentation_*.json dans resultats	JSON_DIR pointe vers resultats/	Corriger JSON_DIR = "outputs" dans le 11
torch >= 2.4 introuvable	Environnement en osx-64 (Intel)	Recréer sous Miniforge arm64 (section 2.1)
Ollama inaccessible	ollama serve non lancé	ollama serve (arrière-plan)
Réponses tronquées A/B/C	MAX_TOKENS trop bas	Monter à 4500 (A/B) ou 5000 (C)
WeasyPrint échoue (macOS)	Pango manquant	brew install pango cairo
ref_courte absent dans sorties	03 non relancé après 00_zotero_import	Relancer 03_build_embeddings_local.py

10. Inventaire complet des scripts

Scripts communs aux deux voies

Script	Voie	Rôle
00a_html_to_pdf.py	A (Zotero)	Convertit snapshots HTML Zotero en PDF
00_zotero_import.py	A (Zotero)	Import initial : pdfs/ + metadata_refs.json
00b_zotero_update.py	A (Zotero)	Mise à jour incrémentale sans rebuild
01_extract_text.py	A + B	Extraction texte des PDFs
02_chunk_corpus.py	A + B	Découpage en chunks avec chevauchement
11_zones_prioritaires.py	A + B	Croisement 09×10, coût zéro, navigation

Scripts voie locale (Ollama)

Script	Entrée	Sortie
03_build_embeddings_local.py	chunks.json	vector_store_local/
04_rag_query_local.py	Question interactive	Réponse via Ollama
05_rag_write_local.py	Consigne de rédaction	redaction_*.txt
05_confronter_local.py	Paragraphe existant (fichier ou console)	confrontation_local_{ts}.txt
06_map_enrich_local.py	manuscript.txt	map_{ts}.json + map_{ts}.md
07_map_critique_local.py	map_{ts}.json	critique.json + critique.md
09_map_argumentation_local.py	map_{ts}.json	argumentation_{ts}.json + .md
10_map_perelman_local.py	map_{ts}.json	perelman_{ts}.json + .md
A_toulmin_segment_local.py	Segment (fichier ou console)	toulmin_{ts}.md + .json
B_perelman_segment_local.py	Segment (fichier ou console)	perelman_seg_{ts}.md + .json
C_rst_segment_local.py	Segment (fichier ou console)	rst_{ts}.md + .json
D_synthese_croisee_local.py	JSON A+B+C	synthese_croisee_{ts}.md